

Database Deletion Operations

Suppose you have this table:

Student

ID	Name	CGPA
1	Aadya	7.97
2	Arpit	7.70
3	Bhaar	8.92

Now imagine different situations.

Situation 1

The college says
Remove Bhaar only.

Use **DELETE**

Situation 2

The college says
New semester starts. Remove every student record. Keep the table.

Use **TRUNCATE**

Situation 3

The college says
Student management system no longer exists. Remove the table itself.

Use **DROP TABLE**

Situation 4

The college closes forever. Remove the whole database.

Use **DROP DATABASE**

DELETE

DELETE removes rows from a table.
The table itself remains.

Syntax

```
DELETE FROM Student;  
DELETE FROM Student WHERE ID = 2;  
DELETE FROM Student;
```

Deletes every row.

The table structure stays.

Columns stay.

Indexes stay.

Constraints stay.

Only data disappears.

Internal Working

When MySQL executes

```
DELETE FROM Student WHERE ID=2;
```

It generally:

- Finds matching rows.
- Marks those rows for deletion.
- Records changes in transactional logs (InnoDB).
- Updates indexes.
- Frees space for future use

Logging

DELETE is fully logged in InnoDB.

Every deleted row is recorded so the operation can be rolled back if it hasn't been committed.

Rollback Example

```
START TRANSACTION;  
DELETE FROM Student WHERE ID=2;  
ROLLBACK;
```

Suppose

1

2

3

Delete

3

Next insert becomes

4

DELETE does not reset the AUTO_INCREMENT counter.

TRUNCATE TABLE

TRUNCATE removes all rows.

Syntax

```
TRUNCATE TABLE Student;
```

The table still exists.

For InnoDB, MySQL implements TRUNCATE TABLE by effectively dropping and recreating the table internally, which makes it very fast for removing all rows.

The table definition remains, but all data is removed.

Unlike DELETE, TRUNCATE does not log each deleted row individually.

It logs the metadata operation instead.

This is one reason it's much faster for clearing an entire table.

Suppose IDs are

1

2

3

Run

```
TRUNCATE TABLE Student;
```

Insert again.

ID becomes

1

The AUTO_INCREMENT counter is reset.

```
START TRANSACTION;  
TRUNCATE TABLE Student;  
ROLLBACK;
```

The rollback does not restore the data.

Unlike DELETE, TRUNCATE cannot be rolled back in the usual transactional sense

Foreign Key Restriction

If another table references Student with a foreign key,

```
TRUNCATE TABLE Student;
```

may fail.

MySQL prevents truncating a table that is referenced by a foreign key.

DROP TABLE

DROP removes

- data
- table structure
- indexes
- constraints
- triggers associated with the table

Everything.

Internal Working

MySQL:

- removes metadata,
- deletes the table definition,
- releases associated storage,
- removes indexes and constraints tied to the table.

In normal MySQL usage, it cannot be rolled back.

No table exists anymore.

So the counter disappears with the table.
If the table is recreated later, it starts fresh

Comparison Table

DELETE	TRUNCATE
Removes selected or all rows	Removes all rows only
Supports WHERE	No WHERE
Row-by-row operation	Table-wide operation
Fully logged row deletions	Metadata-level operation
Can usually be rolled back (before commit)	Implicit commit; not normally rollbackable
Does not reset AUTO_INCREMENT	Resets AUTO_INCREMENT
Generally slower for large tables	Generally much faster

Which is fastest?

Generally: **DROP > TRUNCATE > DELETE**

Which can be rolled back?

DELETE → Yes, before COMMIT in a transaction.

TRUNCATE → No (implicit commit in MySQL).

DROP → No (implicit commit in MySQL).